21 — Relationale Datenbanken

Tabellen, Schlüssel, Beziehungen, Datentypen, ACID

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Warum verknüpfte Tabellen statt einer Riesentabelle?
2. Tabelle = Relation – die Mathematik dahinter
3. Primärschlüssel und NULL
4. Fremdschlüssel und Beziehungstypen
5. SQL-Datentypen – die Wertebereiche
6. Transaktionen & ACID – die Sicherheitsgarantien
7. **Klassiker: Normalisierung der Chaos-Tabelle** – live zerlegt

Lernziel: das relationale Modell **verstehen** – welche Regeln ein DBMS erzwingt und *warum* damit die Anomalien aus Foliensatz 20 unmöglich werden.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Selbst überlegen* (Sie, Notebook 21).

Eine Riesentabelle? Lieber nicht.

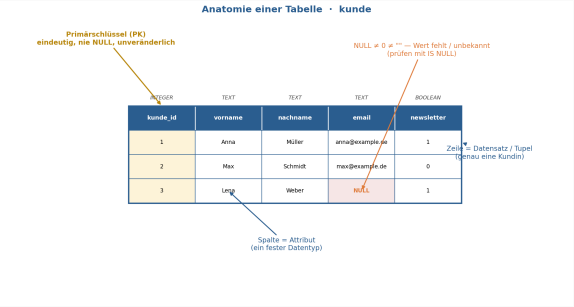
Letzte Woche (FS 20) die **Diagnose** – alles in einer Tabelle:

BestellNr	Kunde	E-Mail	Produkt	Preis
1	Anna Müller	anna@example.de	Eco-Sneaker	89.95
2	Anna Müller	anna@example.de	Hemp-High	109.00
3	Max Schmidt	max@example.de	Hemp-High	109.00

- Annas E-Mail steht **mehrfach** $\rightarrow$ Update-Anomalie beim Ändern
- Produkt + Preis wiederholen sich $\rightarrow$ zwei Versionen desselben Fakts

Idee des relationalen Modells: jede „Sache” genau **einmal** in ihrer eigenen Tabelle – verbunden über **Schlüssel**. Heute lernen wir die Regeln, am Ende **operieren** wir die Chaos-Tabelle.

Anatomie einer Tabelle



- **Zeile** (Datensatz / Tupel) = genau ein Ding · **Spalte** (Attribut) = eine Eigenschaft mit *einem* festen Datentyp
- keine zwei identischen Zeilen, eindeutige Spaltennamen, Zeilenreihenfolge egal

▶ **Live-Demo** – `01_tabelle_als_datenstruktur.py`

Unter der Haube: eine Tabelle ist eine Relation

Jede Spalte i hat einen **Wertebereich** (Domäne) D_i – etwa D_1 = ganze Zahlen, D_2 = Texte. Eine Tabelle ist eine **Teilmenge des Kreuzprodukts**:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

- jede **Zeile** ist ein Tupel $(d_1, \ldots, d_n)$ mit $d_i \in D_i$ – z. B. $(1,\ \text{Eco-Sneaker},\ 89.95)$
- weil R eine **Menge** ist: keine doppelten Zeilen, Reihenfolge bedeutungslos
- jeder Wert ist **atomar** – keine Listen in einer Zelle

„Relational" kommt von **Relation** (Mengenlehre, Mathe 1) – nicht von „Beziehung". Die **Datentypen** (gleich mehr) sind genau diese D_i.

Primärschlüssel – jede Zeile eindeutig

Ein **Primärschlüssel** (PK) ist eine Spalte (oder Kombination), die jede Zeile **eindeutig identifiziert**:

- **eindeutig** – kein Wert kommt zweimal vor
- **nie NULL** – jede Zeile *muss* einen haben
- **unveränderlich** – ändert sich nach dem Anlegen nicht

Gute Kandidaten: Matrikelnummer, ISBN, automatisch hochgezählte ID. Schlechte: Name, E-Mail – ändern sich und sind nicht garantiert eindeutig.

✎ **Selbst überlegen** – Notebook 21, Übung 1.1: Tabelle „Bücher" entwerfen – welche Spalte wird PK?

Unter der Haube: was der PK wirklich erzwingt

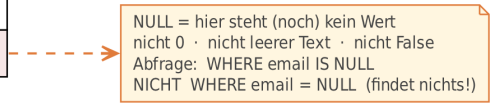
Der PK ist eine **Invariante**: für je zwei verschiedene Zeilen gilt $r \neq s \Rightarrow \text{pk}(r) \neq \text{pk}(s)$ – und das DBMS prüft sie bei **jedem** Einfügen:

Schritt	Einfügen in kunden (konzeptuell)	DBMS-Prüfung
1	Zeile (1, Anna Müller, …)	ok – 1 ist frei
2	Zeile (2, Max Schmidt, …)	ok – 2 ist frei
3	Zeile (2, Lena Weber, …)	**abgelehnt** – 2 existiert schon

FS 20: „Prüfregeln, die Excel nicht hat" – das hier ist so eine. Excel nähme Zeile 3 kommentarlos an; das DBMS weist die **ganze Operation** zurück, die Tabelle bleibt sauber.

NULL – der Wert, der fehlt

kunde_id	name	email
1	Anna Müller	anna@example.de
2	Max Schmidt	max@example.de
3	Lena Weber	NULL



- mögliche Bedeutungen: *unbekannt* · *nicht anwendbar* · *noch nicht eingetragen* – das Python-Gegenstück ist `None`
- **nie** beim Primärschlüssel; mit `NOT NULL` macht man eine Spalte zur Pflicht

Unter der Haube: NULL ist kein Wert

NULL ist eine **Markierung** („hier fehlt etwas”), kein Element des Wertebereichs D_i. Deshalb rechnet ein DBMS mit NULL anders:

Ausdruck	Ergebnis
`0 = 0`	wahr
`NULL = 0`	*nicht* wahr – unbekannt
`NULL = NULL`	**nicht wahr!** – zwei Unbekannte

- zwei *unbekannte* Telefonnummern sind nicht automatisch dieselbe Nummer – darum ist `NULL = NULL` nicht wahr
- Vorgeschmack: **dreiwertige Logik** (wahr / falsch / unbekannt) – in NB 23 prüfen wir deshalb mit `IS NULL`, nie mit `= NULL`

Fremdschlüssel – Tabellen verbinden

Ein **Fremdschlüssel** (FK) ist eine Spalte, die auf den **Primärschlüssel einer anderen Tabelle** zeigt:

bestell_id (PK)	kunden_id (FK)	betrag
10	1	89.95
11	1	109.00
12	2	135.50

- `kunden_id` zeigt in die Tabelle kunden (1 = Anna Müller, 2 = Max Schmidt) – Annas Daten werden **referenziert, nicht kopiert**

▶ **Live-Demo** – `02_fremdschluessel_beziehung.py` (verwaiste Bestellung aufspüren)

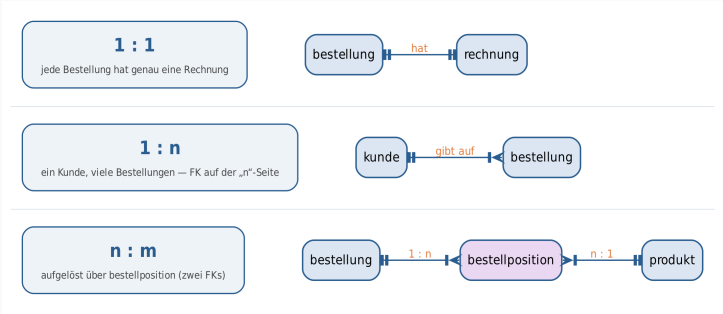
Unter der Haube: referenzielle Integrität

Die FK-Invariante: **jeder FK-Wert muss als PK-Wert in der Zieltabelle existieren** – geprüft beim Einfügen, Ändern *und* Löschen:

Schritt	Aktion (konzeptuell)	DBMS
1	Bestellung mit kunden_id = 2 anlegen	ok – Kunde 2 existiert
2	Bestellung mit kunden_id = 99 anlegen	**abgelehnt** – kein Kunde 99
3	Kunde 2 löschen	**abgelehnt** – Bestellung 12 zeigt auf ihn

Schritt 3 ist der Clou: die **Delete-Anomalie** aus FS 20 wird damit *strukturell unmöglich* – kein FK darf je ins Leere zeigen, also gibt es keine verwaisten Datensätze.

Drei Beziehungstypen



Der **Fremdschlüssel steht auf der „n"-Seite**. Eine **n:m**-Beziehung braucht eine **Zwischentabelle** mit zwei Fremdschlüsseln.

✎ **Selbst überlegen** – Notebook 21, Übung 2.1: Beziehungen für ein Shop-System modellieren.

Unter der Haube: warum n:m eine Zwischentabelle braucht

- Ein FK-Feld hält genau **einen** Wert → eine Zeile kann nur auf **einen** Partner zeigen → direkt geht höchstens 1:n
- Eine *Liste* von IDs in die Zelle schreiben? Verboten – Werte einer Relation sind **atomar** (Kreuzprodukt-Regel!)
- Lösung: jede Zeile der Zwischentabelle speichert **ein Paar** aus zwei FKs

bestell_id (FK)	produkt_id (FK)	menge
10	1	1
10	2	1
11	3	2

n:m ist also nichts Neues, sondern **zwei 1:n-Beziehungen**, die sich in der Zwischentabelle treffen.

SQL-Datentypen – die richtige Schublade

Kategorie	Typen	wofür
Ganzzahl	`INTEGER`, `BIGINT`	IDs, Stückzahlen
Dezimal	`DECIMAL(p,s)`	**Geldbeträge** – exakt
Gleitkomma	`FLOAT`, `DOUBLE`	Messwerte (nicht für Geld!)
Text	`CHAR(n)`, `VARCHAR(n)`, `TEXT`	feste / variable / lange Texte
Datum/Zeit	`DATE`, `TIME`, `DATETIME`, `TIMESTAMP`	Termine, Zeitstempel
weitere	`BOOLEAN`, `ENUM`, `BLOB`	Ja/Nein, feste Auswahl, Binärdaten

Unter der Haube: Datentypen sind die Wertebereiche

- Der Typ einer Spalte **ist** ihr D_i aus der Relationsformel: `DECIMAL(6,2)` bedeutet $D_{\text{preis}} = \{-9999.99, \ldots, 9999.99\}$ in Hundertstel-Schritten
- Damit ist der Typ eine **Prüfregel** wie der PK: Wert außerhalb von D_i $\rightarrow$ Einfügen abgelehnt
- `DECIMAL` rechnet **exakt** in Zehner-Brüchen; `FLOAT` ist IEEE 754 binär – NB 05: `0.1 + 0.2` ergibt nicht exakt `0.3` $\rightarrow$ **nie Geld als FLOAT**
- Telefonnummern als **Text**: führende Null und +49 gehen als Zahl verloren

✎ **Selbst überlegen** – Notebook 21, Übung 3.1: für E-Mail, Preis, Lagerbestand, Geburtsdatum … den optimalen Typ wählen.

Transaktion – alles oder nichts

Eine Bestellung bei Veggie Soles ist **ein** Vorgang aus **drei** Schritten:

1. Lagerbestand senken
2. Bestellung anlegen
3. Rechnung schreiben

- Eine **Transaktion** ist die Klammer darum: ausgeführt werden **alle** Schritte – oder **keiner**
- Abschluss: **COMMIT** („alles gilt jetzt") oder **ROLLBACK** („nichts gilt – Datenbank wie vorher")

Absturz nach Schritt 1? *Ohne* Transaktion: Ware weg, aber keine Bestellung. *Mit* Transaktion: automatischer ROLLBACK – als wäre nichts passiert.

ACID – die vier Garantien

	steht für	bedeutet
A	Atomicity	alles oder nichts – keine halben Transaktionen
C	Consistency	alle Regeln (PK, FK, Typen) bleiben gültig
I	Isolation	parallele Transaktionen stören sich nicht
D	Durability	bestätigte Änderungen überleben jeden Absturz

Ohne ACID kein Online-Banking, kein E-Commerce, kein Ticketverkauf. Genau das unterscheidet ein echtes DBMS von einer Datei.

Unter der Haube: ACID im Veggie-Soles-Alltag

	ein konkretes Szenario
A	Absturz zwischen Lagerbuchung und Bestellung → beides zurückgerollt – nie ein halber Vorgang
C	Bestellung mit kunden_id = 99 → ganze Transaktion abgelehnt – die FK-Invariante wird **nie** verletzt
I	zwei Kund:innen, das **letzte** Paar Bambus-Boot: eine Transaktion gewinnt, die andere sieht „ausverkauft" – nie beide
D	„Bestellung bestätigt" steht **vor** der Bestätigung auf der Platte – FS 12: erst wenn der **Puffer** geleert ist, ist es sicher

✎ **Selbst überlegen** – Notebook 21, Übung 4.1: was ginge bei einer Shop-Bestellung ohne jede der vier Garantien schief?

Klassiker: Normalisierung der Chaos-Tabelle – die Aufgabe

FS 20 war die **Diagnose** (vier Anomalien) – heute **operieren** wir:

BestellNr	Kunde	E-Mail	Produkt	Preis	Menge
1	Anna Müller	anna@example.de	Eco-Sneaker	89.95	1
2	Anna Müller	anna@example.de	Hemp-High	109.00	1
3	Max Schmidt	max@example.de	Hemp-High	109.00	1
4	Anna Müller	anna@example.de	Bambus-Boot	135.50	2
5	Max Schmidt	max@example.de	Eco-Sneaker	89.95	1
6	Anna Müller	anna@example.de	Eco-Sneaker	89.95	1

- Zerlegen Sie in **drei** Tabellen: kunden, `produkte`, `bestellungen`
- Je Tabelle den **PK** bestimmen – `bestellungen` bekommt zwei **FKs**
- **Gegenprobe**: welche der vier Anomalien funktioniert noch?

Klassiker: Normalisierung – Live-Audit

▶ **Live-Audit am Beamer** – Referenz: `90_klassiker_loesung.md`
Teilziele:

1. Wiederholungsgruppen identifizieren – welche Fakten stehen mehrfach da?
2. `kunden` herauslösen + künstlichen PK `kunden_id` vergeben
3. `produkte` herauslösen + PK `produkt_id` vergeben
4. `bestellungen` als Verbindung: PK `bestell_id` + **zwei FKs**
5. Gegenprobe: alle vier Anomalien aus FS 20 durchgehen – keine funktioniert mehr

✎ **Zum Selberlösen** – Handout *“Klassiker 21: Normalisierung”* (`Klassiker/21_Klassiker_Normalisierung.pdf`). Vertiefung: Notebook 21, Übung 2.1 und Abschlussaufgabe 2.

Cheat Card

Begriff	Kurz
Tabelle / Zeile	Relation $R \subseteq D_1 \times \cdots \times D_n$ / Tupel
PK / FK	eindeutig + nie NULL / muss als PK der Zieltabelle existieren
NULL	kein Wert – `NULL = NULL` ist *nicht* wahr
1:1 / 1:n / n:m	FK auf der n-Seite – n:m braucht Zwischentabelle
`DECIMAL(p,s)`	exakter Typ für Geld (nie `FLOAT`)
Transaktion / ACID	alles-oder-nichts / die vier Garantien
Klassiker heute	Chaos-Tabelle $\rightarrow$ `kunden` + `produkte` + `bestellungen`

Faustregel: erst das Datenmodell, dann der Code – jede Regel im Modell verteidigt das DBMS ab dann **automatisch**.

Ausblick: ER-Diagramme (21b) und SQLite (NB 22)

Unsere drei Tabellen existieren bisher nur **auf Papier** – genau da geht es weiter:

- **Foliensatz 21b** – das **Entity-Relationship-Diagramm**: Datenmodelle *zeichnen*, bevor man eine Tabelle anlegt
- **Notebook 22** – SQLite mit Python: die Tabellen wirklich bauen, füllen, abfragen

Nur ansehen – diese Syntax lernen wir erst in NB 22:

```
CREATE TABLE produkte (
    produkt_id INTEGER PRIMARY KEY,
    name  TEXT,
    preis REAL);
```

Heute gelernt

- ✓ Riesentabelle $\rightarrow$ verknüpfte Tabellen: Redundanz raus, Schlüssel rein
- ✓ Tabelle = Relation $R \subseteq D_1 \times \cdots \times D_n$
- ✓ PK als Invariante – bei jedem Einfügen geprüft
- ✓ NULL ist kein Wert – `NULL = NULL` ist nicht wahr
- ✓ FK + referenzielle Integrität – Delete-Anomalie unmöglich
- ✓ 1:1 / 1:n / n:m – n:m braucht die Zwischentabelle
- ✓ Datentypen = Wertebereiche · Transaktionen & ACID
- ✓ **Klassiker Normalisierung** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Normalisierung** eigenständig lösen (Handout, Papier genügt)
- „Selbst überlegen"-Übungen 1.1–4.1 im Notebook 21
- Abschlussaufgaben NB 21 (bis zum Ticketsystem-Schema)